

Unit-4

Object-oriented programming (OOP)

Object-oriented programming (OOP) :

- Object-oriented programming (OOP) allows to model real-world entities in code, making programs more organized, reusable and easier to maintain.
- By grouping related data and behavior into a single unit, classes and objects help write cleaner, more logical code for everything from small scripts to large applications.

Class:

- A class is like a blueprint or template.
or
- A class is a user-defined template for creating objects.
- It bundles data and functions together, making it easier to manage and use them.
- When we create a new class, we define a new type of object. We can then create multiple instances of this object type.

Creating Class

- Classes are created using class keyword. Attributes are variables defined inside class and represent properties of the class.

Ex:

define a class

class Dog:

 sound = "bark" # class attribute

Object

- An object is a specific instance of a class. It holds its own set of data (instance variables) and can invoke methods defined by its class.
- Multiple objects can be created from same class, each with its own unique attributes.

Create an object:

Ex:

class Dog:

 sound = "bark"

dog1 = Dog() # Creating object from class

print(dog1.sound) # Accessing the class

Constructor method:

- In Python, a constructor is a special method that is called automatically when an object is created from a class.
- Its main role is to initialize the object by setting up its attributes or state.
- The method `__new__` is the constructor that creates a new instance of the class while `__init__` is the initializer that sets up the instance's attributes after creation.
- These methods work together to manage object creation and initialization.

__init__ method:

- This method initializes the newly created instance and is commonly used as a constructor in Python.
- It is called immediately after the object is created by `__new__` method and is responsible for initializing attributes of the instance.

Syntax:

```
class ClassName:
    def __new__(cls, parameters):
        instance = super(ClassName, cls).__new__(cls)
        return instance
```

__new__ Method:

- This method is responsible for creating a new instance of a class. It allocates memory and returns the new object. It is called before `__init__`.

Syntax:

```
class ClassName:
    def __new__(cls, parameters):
        instance = super(ClassName, cls).__new__(cls)
        return instance
```

Differences Between `__init__` and `__new__`

__new__ method:

- Responsible for creating a new instance of the class.
- Rarely overridden but useful for customizing object creation and especially in singleton or immutable objects.

__init__ method:

- Called immediately after `__new__`.
- Used to initialize the created object.

Types of Constructor:

There are 2 types are

1. Default Constructor

A **default constructor** does not take any parameters other than **self**. It initializes the object with default attribute values.

Ex:

class Car:

```
def __init__(self):
```

```
    #Initialize the Car with default attributes
```

```
    self.make = "Toyota"
```

```
    self.model = "Corolla"
```

```
    self.year = 2020
```

```
# Creating an instance using the default constructor
```

```
car = Car()
```

```
print(car.make)
```

```
print(car.model)
```

```
print(car.year)
```

output:

Toyota

Corolla

2020

2. Parameterized Constructor

A **parameterized constructor** accepts arguments to initialize the object's attributes with specific values.

Ex:

class Car:

```
def __init__(self, make, model, year):
```

```
    #Initialize the Car with specific attributes.
```

```
    self.make = make
```

```
self.model = model
```

```
self.year = year
```

```
# Creating an instance using the parameterized constructor
```

```
car = Car("Honda", "Civic", 2022)
```

```
print(car.make)
```

```
print(car.model)
```

```
print(car.year)
```

output:

Honda

Civic

2022

Classes with multiple objects:

In Python, a class is like a blueprint, and objects are instances created from that blueprint. When you create multiple objects, each object has its own copy of data (attributes), but they all share the same structure (methods and class definition).

Key Concepts of Multiple Objects

- **Independent State:** Every object has its own set of instance variables. For example, if you have a Car class, you can create one object for a "Red Toyota" and another for a "Blue Ford". Changing the color of the first car does not affect the second.
- **Shared Behavior:** All objects created from the same class share the same methods (functions). Both car objects will have access to the same .drive() or .brake() methods defined in the class.
- **Instantiation:** You create multiple objects by calling the class name as if it were a function multiple times, typically assigning each to a different variable.

Ex:

```
class Car:
```

```
    def __init__(self, brand, color):
```

```
        self.brand = brand
```

```
        self.color = color
```

```
    def show(self):
```

```
        print(self.brand, "-", self.color)
```

```
# Multiple objects
```

```
c1 = Car("Toyota", "Red")
```

```
c2 = Car("Honda", "Blue")
```

```
c1.show()
```

```
c2.show()
```

objects as arguments:

In Python, objects can be passed as arguments to functions or methods, just like normal variables (integers, strings, etc.). Since everything in Python is an object, you can pass entire objects and work with their data inside functions.

“Passing an object as an argument means sending one object to another object’s method so it can use its data.”

Ex:

```
class Student:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def compare(self, other):
```

```
        if self.name == other.name:
```

```
            print("Same name")
```

```
        else:
```

```
            print("Different name")
```

```
# Create objects
```

```
s1 = Student("ram")
```

```
s2 = Student("Rahul")
```

```
# Pass object as argument
```

```
s1.compare(s2)
```

objects as a return value:

- Returning an object means a function sends back a complete object so it can be used later.
Or
- Object as a return value means a function or method returns a complete object instead of a normal value like number or string.

Ex:

```
class Student:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
# Function returning an object
```

```
def get_student():
```

```
    s = Student("Ram") # create object
```

```
    return s           # return object
```

```
# Receive returned object
```

```
s1 = get_student()
```

```
print(s1.name)
```

File Handling in Python:

- File handling refers to the process of performing operations on a file, such as creating, opening, reading, writing and closing it through a programming interface.
- It involves managing the data flow between the program and the file system on the storage device, ensuring that data is handled safely and efficiently.

Why do we need File Handling

- To store data permanently, even after the program ends.
- To access external files like .txt, .csv, .json, etc.
- To process large files efficiently without using much memory.
- To automate tasks like reading configs or saving outputs.

File types:

In Python file handling, files are primarily categorized into two types based on how data is stored and interpreted:

Text Files and Binary Files.

1. Text Files (t):

Text files store data as a sequence of human-readable characters.

- **Structure:** Each line is terminated with a special character known as the End of Line (EOL), which is the newline character (`\n`) in Python by default.
- **Encoding:** Python automatically handles encoding (e.g., ASCII or UTF-8) and decoding between the raw bytes and readable strings.
- **Common Extensions:** .txt, .csv, .py, .json, .html.
- **Usage:** These are typically handled using standard string methods.

2. Binary Files (b)

Binary files store data as raw bytes (0s and 1s) rather than readable characters.

- **Structure:** There is no specific line terminator like EOL. Data represents the actual content exactly as it is stored on the disk.
- **Handling:** Data is read and written as bytes objects without any encoding or decoding conversion.
- **Common Extensions:** .jpg, .png, .mp3, .mp4, .exe, .zip, .pdf.
- **Usage:** Used for non-textual data like images and audio. They require specific software to be interpreted correctly; opening them in a standard text editor often results in "garbage" characters.

Operations on file handling:

Creating a file :

Creating a file in Python means making a new file using a program so that data can be stored permanently.

- Python uses the built-in **open()** function to create a file.

Syntax:

```
open("filename", "mode")
```

ex:

```
file = open("sample.txt", "w")
```

```
file.write("Hello, this is a new file.")
```

```
file.close()
```

File Modes for Creation:

- ☐ "w" → Write mode (creates file if not exists, overwrites if exists)
- ☐ "x" → Create mode (creates new file, gives error if file exists)
- ☐ "a" → Append mode (creates file if not exists, adds data)

Opening a file:

To open a file, we can use [open\(\)](#) function, which requires file-path and mode as arguments.

Syntax:

```
file = open('filename.txt', 'mode')
```

- **filename.txt:** name (or path) of the file to be opened.
- **mode:** mode in which you want to open the file (read, write, append, etc.).

Ex:

```
f = open("geek.txt", "r")  
  
print(f)
```

Closing a File

The `file.close()` method closes the file and releases the system resources. If the file was opened in write or append mode, closing ensures that all changes are properly saved.

EX:

```
file = open("geek.txt", "r")  
  
# Perform file operations  
  
file.close()
```

Reading a File

Reading a file can be achieved by **file.read()** which reads the entire content of the file. After reading, it's good practice to close the file to free up system resources.

Example: Reading a File in Read Mode (r)

```
file = open("geek.txt", "r")  
  
content = file.read()  
  
print(content)  
  
file.close()
```

Writing a File:

In Python, writing to a file is done using the mode "w". This creates a new file if it doesn't exist, or overwrites the existing file if it does. The `write()` method is used to add content. After writing, make sure to close the file.

EX:

```
with open("geek.txt", "w") as file:  
    file.write("Hello, Python!\n")  
    file.write("File handling is easy with Python.")  
  
print("File written successfully")
```


file paths and file names:

In Python, a file path is a string that identifies the location of a file or directory in the file system, while the file name is the specific name of that file, usually including an extension.

Types of File Paths

- **Absolute Path:** The full path starting from the root of the file system (e.g., C:\Users\Name\data.csv on Windows or /home/user/data.csv on Linux/macOS).
- **Relative Path:** A path relative to the **Current Working Directory (CWD)**. For example, if you are already in the Documents folder, the relative path to a file inside it is simply its name, like report.txt.
- **Special Symbols:**
 - . (Single dot): Represents the current directory.
 - .. (Double dot): Represents the parent (one level up) directory.

Handling Paths in Python

Python provides two primary modules for managing paths:

- [pathlib](#) (**Recommended**): An object-oriented approach that is cross-platform. It automatically handles the difference between Windows backslashes (\) and Linux forward slashes (/).

EX:

```
from pathlib import Path
path = Path("folder") / "subfolder" / "file.txt"
print(path.name) # Output: file.txt
print(path.stem) # Output: file
print(path.suffix) # Output: .txt
```

[os.path](#): An older, string-based module for path manipulations.

- os.path.join(): Safely combines path components.
- os.path.basename(): Extracts only the file name from a full path.
- os.path.dirname(): Extracts the directory path.

File Name Components:

A file name typically consists of two main parts:

- **Stem:** The main name of the file (e.g., invoice in invoice.pdf).
- **Extension (Suffix):** The part following the final dot that indicates the file type (e.g., .py, .txt, .csv).

Exception handling :

Exception handling in Python is a mechanism used to manage runtime errors, ensuring that a program does not crash when it encounters unexpected situations. Instead of terminating abruptly, Python allows you to detect the problem, respond to it, and continue execution where possible.

Types of errors:

Errors are problems in a program that causes the program to stop its execution. On the other hand, exceptions are raised when some internal events change the program's normal flow.

Errors in Python generally fall into three main categories: **Syntax Errors**, **Runtime Errors (Exceptions)**, and **Logical Errors**.

1.Syntax Errors in Python:

Often called "parsing errors," these occur when the code violates Python's formal grammar rules. The Python Interpreter detects these before the program even begins to run.

- **IndentationError:** Occurs when code blocks (like those inside an if statement) are not properly indented.
- **Example:** Missing a colon (:) after an if or for statement, or mismatched parentheses

EX:

```
print("Hello"
```

2. Runtime Errors (Exceptions)

These errors occur during program execution even if the syntax is perfect. They usually represent unexpected conditions that crash the program.

Error: Attempting to use a variable or function that has not been defined.

- **TypeError:** Performing an operation on incompatible data types (e.g., adding a string to an integer).
- **IndexError:** Trying to access a list or sequence index that is out of range.
- **KeyError:** Attempting to access a key that does not exist in a dictionary.
- **ZeroDivisionError:** Raised when the second operand of a division or modulo operation is zero.
- **AttributeError:** Trying to access a method or attribute that doesn't exist for a specific object.
- **ImportError / ModuleNotFoundError:** Occurs when an import statement fails to find the specified module

EX:

```
a = 10
```

```
b = 0
```

```
print(a / b)
```

output:

ZeroDivisionError: division by zero

3. Logical Errors

These are the most difficult to detect because the program runs without crashing but produces incorrect results. They are caused by flaws in the programmer's logic or algorithms

EX:

```
a = 10
```

```
b = 5
```

```
print("Sum:", a - b) # wrong logic
```

output:

Sum: 5 # incorrect (should be 15)

Exception:

Problems that occur at runtime and can be managed using exception handling (e.g., invalid input, missing files).

Syntax of Exception Handling

Python provides four main keywords for handling exceptions: try, except, else and finally each plays a unique role. Let's see syntax:

```
try:  
    # Code  
except SomeException:  
    # Code  
else:  
    # Code  
finally:  
    # Code
```

- **try:** Runs the risky code that might cause an error.
- **except:** Catches and handles the error if one occurs.
- **else:** Executes only if no exception occurs in try.
- **finally:** Runs regardless of what happens useful for cleanup tasks like closing files.

Exception handling using try, except, finally:

- **try:** Wraps the "risky" code that might raise an error.
- **except:** Catches and handles the specific exception if one occurs in the try block.

- else (Optional): Executes only if the code in the try block runs successfully without any exceptions.
- finally: Executes unconditionally, regardless of whether an exception was raised, caught, or even if the program terminates. It is primarily used for cleanup tasks like closing files or database connections

EX:

Python code to illustrate working of try()

```
def divide(x, y):
```

```
    try:
```

```
        # Floor Division : Gives only Fractional
```

```
        # Part as Answer
```

```
        result = x // y
```

```
        print("Yeah ! Your answer is :", result)
```

```
    except ZeroDivisionError:
```

```
        print("Sorry ! You are dividing by zero ")
```

Look at parameters and note the working of Program

```
divide(3, 2)
```

```
divide(3, 0)
```

output:

Yeah ! Your answer is : 1

Sorry ! You are dividing by zero

Python finally Keyword

Python provides a keyword [finally](#), which is **always executed** after try and except blocks. The finally block always executes after normal termination of try block or after try block terminates due to some exception. Even if you return in the except block still the finally block will execute.

EX:

Python code to illustrate

working of try()

```
def divide(x, y):
```

```
    try:
```

```
        # Floor Division : Gives only Fractional
```

```
# Part as Answer

result = x // y
except ZeroDivisionError:
    print("Sorry ! You are dividing by zero ")
else:
    print("Yeah ! Your answer is :", result)
finally:
    # this block is always executed
    # regardless of exception generation.
    print("This is always executed")

# Look at parameters and note the working of Program
divide(3, 2)
divide(3, 0)
```